# FUNCTIONS

By: Gretchen Y. Bonilla

Department of Computer Science and Engineering

University of Puerto Rico, Mayaguez

# Objectives



**Introduce functions.**



**How to use them.**



**Introduce pass by reference vs by value**



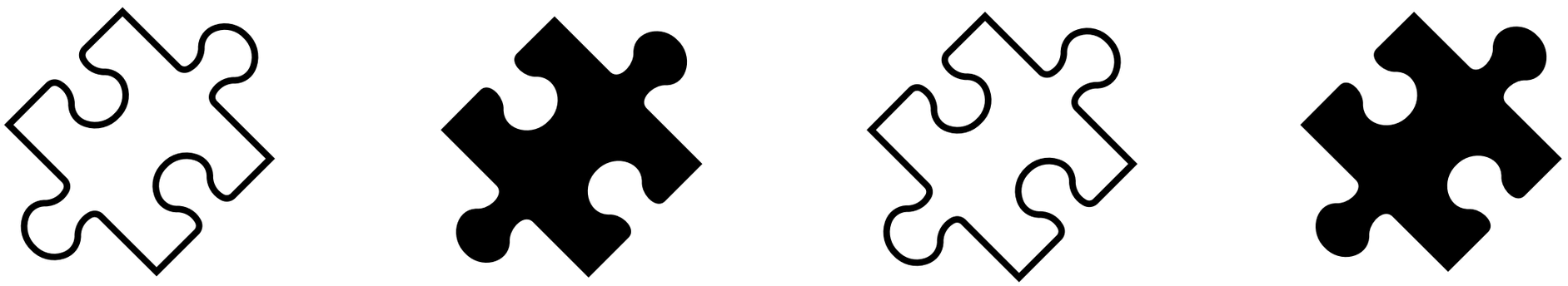
# Modular Programming

Because bite size is fun size

# Modular Programming

## What is Modular Programming?

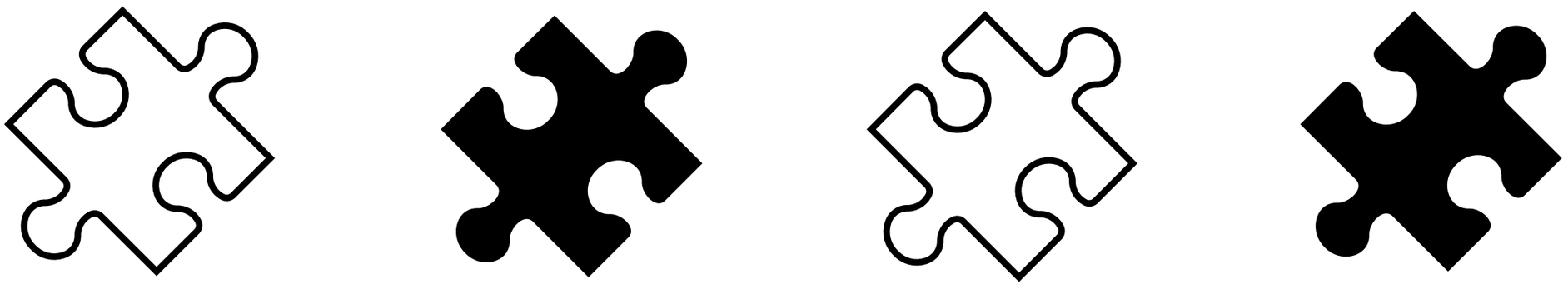
- Dividing a program into ***smaller*, independent modules** (functions, classes, etc.).
- Each module handles a **specific task**.
- Modules can be reused in different parts of the program.



# Modular Programming

## Why use Modular Programming?

- **Simplifies code** → easier to read and debug.
- **Improves maintainability** → changes affect fewer places.
- **Increases reusability** → modules can be plugged into other programs.



# Example

## ***Without*** modular programming

- Notice that, after the `cin`, all three lines are practically identical.
- Difference is the value being checked and part of the being message printed.
- If we want to make changes to the condition or message, we have to updated all three lines.

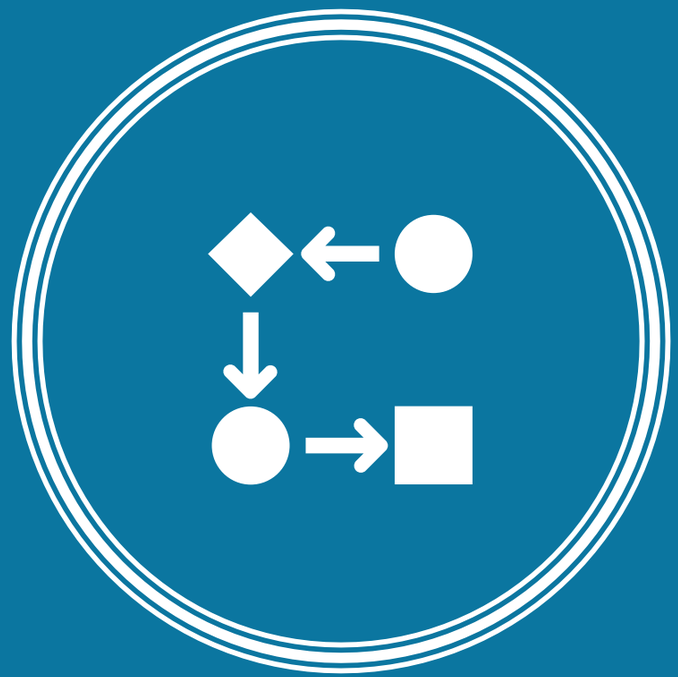
```
int main() {
    int x, y, z;
    cin >> x >> y >> z;
    if(x % 2 == 0) cout << "X is even" << endl;
    if(y % 2 == 0) cout << "Y is even" << endl;
    if(z % 2 == 0) cout << "Z is even" << endl;
}
```

# Example

## ***With*** modular programming

- The function `checkIsEven` captures all the recurring code and receives as parameters the parts that are different.
- If we want to make changes, we only need to adjust the function and it applies to everyone.
- Makes code easier to read.

```
void checkIsEven(int value, string name) {
    if(value % 2 == 0) cout << name << “ is even” << endl;
}
int main() {
    int x, y, z;
    cin >> x >> y >> z;
    checkIsEven(x, "X");
    checkIsEven(y, "Y");
    checkIsEven(z, "Z");
}
```



# Function Definition

# Functions

- A group of statements that does a *specific* task.
- It can:
  - Receive input (parameters)
  - Do a process
  - Return an output (result/value)

# Type of Functions

- **Value returning** function: Returns a result
- **Void** function: doesn't return anything

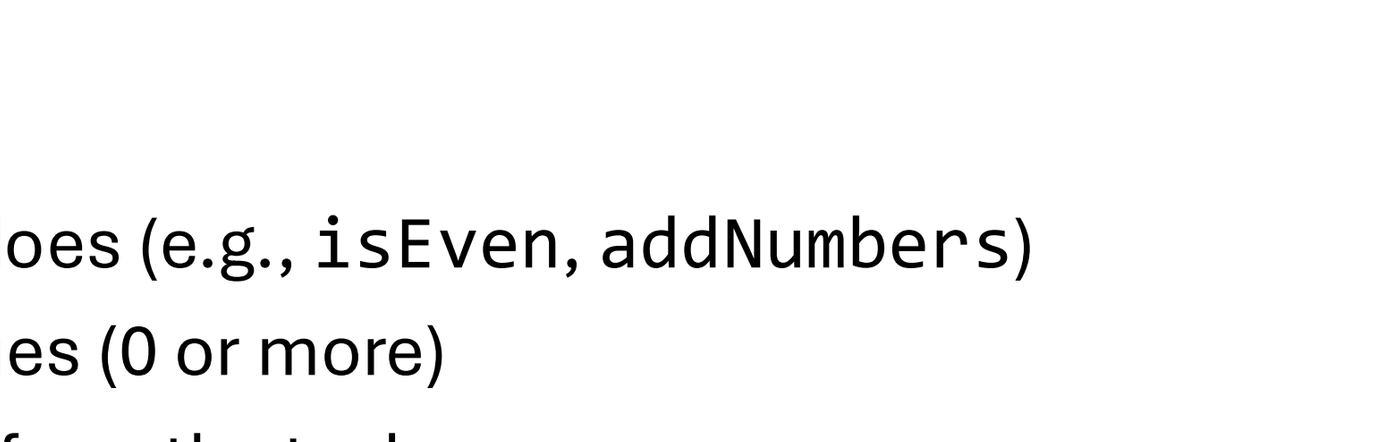
```
// Value-returning functions
bool isEven(int value) { return value % 2 == 0; }

// Another example
int addNumbers(int a, int b) {
    return a + b;
}
// Void function (no return)
void sayHello() {
    std::cout << "Hello!" << std::endl;
}
```

# Function Definition

**A function consists of:**

- **Name** – describes what it does (e.g., `isEven`, `addNumbers`)
- **Parameter List** – input values (0 or more)
- **Body** – statements that perform the task
- **Return Type** – type of result (`int`, `bool`, `void`, etc.)



# Function Definition

**Name**

- Should be indicative of the task it will do.
- Usually includes verbs.
- Follows same naming conventions as variables.
  - `camelCase`
  - `snake_case`
  - Shouldn’t start with numbers or use special characters

Name

```
bool isEven(int value) {
    return value % 2 == 0;
}
```

# Function Definition

## Parameter List

- Data that the function *needs*.
- Can have from 0 to more parameters.
- Not required to have any.
- Type *must* be indicated per parameter.

1 Parameter

```
bool isEven(int value) {
    return value % 2 == 0;
}
```

2 Parameters

```
int addTriangles(Triangle t1, Triangle t2);
```

No Parameters

```
double getAngle() {
    return this->angle;
}
```

# Function Definition

## Body

- Collection of statements that do a task.
- Ideally, a function should do *one* specific task.

```
bool isEven(int value) {
    return value % 2 == 0;
}
```

Body

# Function Definition

## Return Type

- The result from the body should be of this type.
- What we expect from the function.
- We use the **`return`** keyword.
- If it *doesn't* return anything the type is **`void`**.

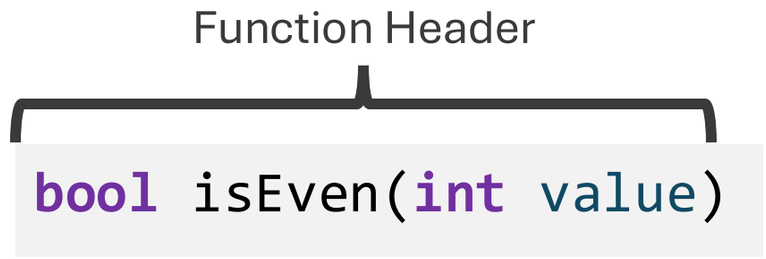
**Return type**

```
bool isEven(int value) {
    return value % 2 == 0;
}
```

# Function Header

## Function Header

- Consists of:
  - Return type
  - Name
  - Parameter List
- Indicates *bare minimum* information of the function.



# Function Calls

```
bool isEven(int value) {
    return value % 2 == 0;
}

int main() {
    int x = 15;
    bool result = isEven(x);
    cout << "Result: " << result << endl;
}
```

**Declaration and definition**

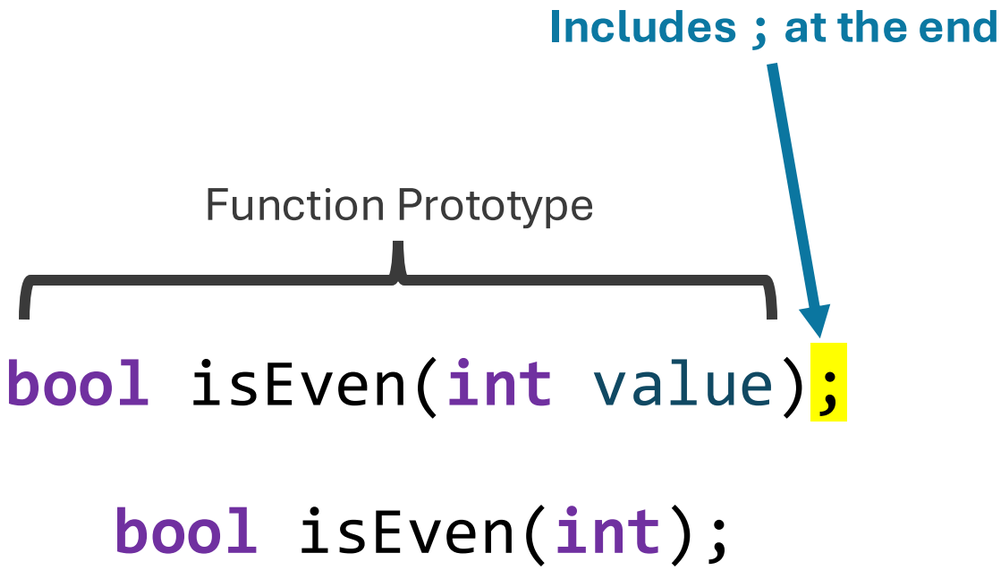
**Call**

## Function Calls

- It's when we *use* the function.
- Only the name and in between the parenthesis we pass the parameters.
- Statement ends with a ;

# Function Prototypes

- A declaration of a function that appears *before* its use.
- Allows us to call a function *before* its definition.
- Consists of the **function header + semicolon (;)**.
- Parameter names are optional, but types are required.



# Function Prototypes

**We cannot call a function that has not declared prior.**

```
int main() {
    int x = 15;
    bool result = isEven(x);
    cout << "Result: " << result <<endl;
}
```

**Will throw an error since `isEven` has not been declared or defined.**

# Function Prototypes

**We cannot call a function that has not declared prior.**

```
int main() {
    int x = 15;
    bool result = isEven(x);
    cout << "Result: " << result <<endl;
}
bool isEven(int value) {
    return value % 2 == 0;
}
```

**Will *still* throw an error since `isEven` has not been declared *prior* to being called.**

# Function Prototypes

**We cannot call a function that has not declared prior.**

```
bool isEven(int value) {
    return value % 2 == 0;
}

int main() {
    int x = 15;
    bool result = isEven(x);
    cout << "Result: " << result <<endl;
}
```

**Will work, `isEven` is defined prior to being used.**

# Function Prototypes

**We cannot call a function that has not declared prior.**

```
bool isEven(int);
int main() {
    int x = 15;
    bool result = isEven(x);
    cout << "Result: " << result <<endl;
}
bool isEven(int value) {
    return value % 2 == 0;
}
```

**Will also work. `isEven` is declared prior to being used. Even if its formal definition is afterwards.**

**This allows for better organization.**



# Parameter Lists

The information provided by the user; be wary.

# Parameters

- Functions can take **0 or more inputs**.
- Inputs are called **parameters or arguments**.

**Types**:

- **Primitives** → `int`, `double`, `char`
- **User-defined** → `struct`, `class`, `enum`

```
// Primitive parameter
bool isEven(int value) {
    return value % 2 == 0;
}
```

```
// User-defined parameter
struct Dog {
    int age;
    string name;
};
int getDogYears(Dog dog) {
    return dog.age * 7;
}
```

# Parameter Types

Function calls can receive literals, variables and/or expressions.

```
bool isEven(int value) {
    return value % 2 == 0;
}
```

**Literal**

```
bool result = isEven(12);
```

**Variable**

```
int x = 98;
bool result = isEven(x);
```

**Expression**

```
int y = 0;
int x = 98;
bool result = isEven((y + x)* 2);
```

# Pass by Value vs Reference

- **Pass by value** (default behavior): function gets a *copy* → original stays unchanged.
- **Pass by reference** (&): function works *directly* on the variable → changes persist.

```
// Pass by value
void increaseValue(int x) { x++; }

int main() {
    int a = 10;
    increaseValue(a); // a is still 10
}
```

```
// Pass by reference
void increaseValue(int &x) { x++; }

int main() {
    int b = 10;
    increaseValue(b); // b is now 11
}
```

# Pass by Value

- The parameter is a *copy* of what the function call passed it.
- Changes to the parameter will *not* reflect outside of the method (changes stay local).
- *By default*, all parameters are passed by value.

```
void increaseValue(int x) {
    x++;
    cout << "Value after increase: " << x << endl;
}

int main() {
    int x = 10;
    increaseValue(x);
    cout << "Value after increase method: " << x << endl;
}
```

**Console Output:**

```
Value after increase: 11
Value after increase method: 10
```

# Pass by Reference

- The parameter is the memory location of the value passed in the function call.
- Changes to the parameter reflect outside of the method.
- The parameters need to be a pointer, which is indicated using & after the parameter type.

& indicates it's a memory location

```
void increaseValue(int &x) {
    x++;
    cout<< "Value after increase: " << x << endl;
}
int main() {
    int x = 10;
    increaseValue(x);
    cout<< "Value after increase method: " << x << endl;
}
```

**Console Output:**
```
Value after increase: 11
Value after increase method: 11
```

# Value vs Reference

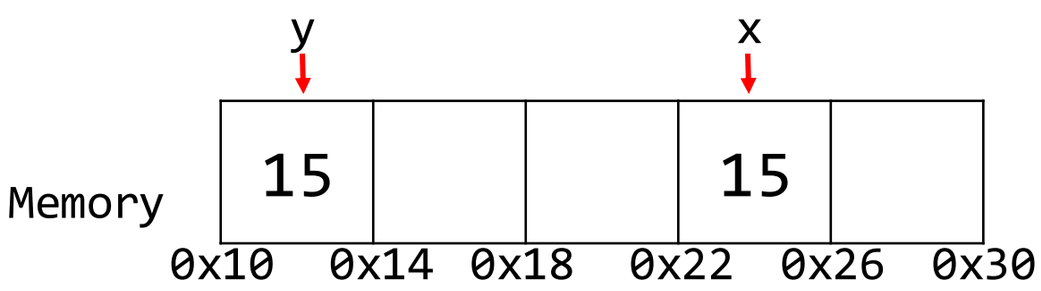
**Value**

- Uses twice the memory for a single value (need to store copy).
- But keeps original safe.

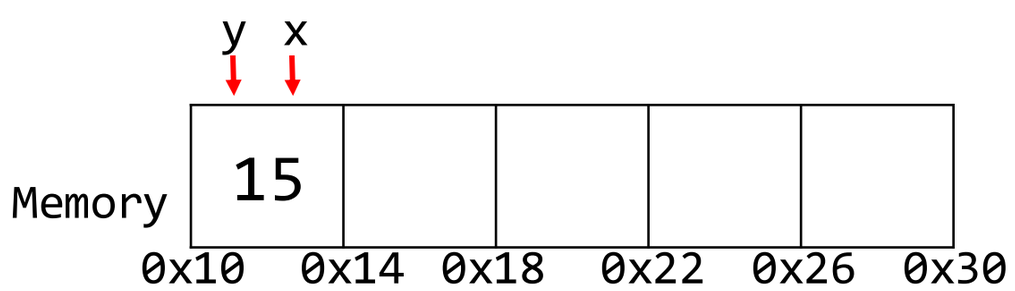
**Reference**

- Uses same value, wasting less memory.
- Changes will reflect outside of function.

```
void increaseValue(int x) {x++;}
int y = 15;
increaseValue(y);
```




```
void increaseValue(int &x) {x++;}
int y = 15;
increaseValue(y);
```



# Const Reference

- References are useful for saving memory but allows users to modify the value.
- Using the `const` keyword for the parameter can avoid this.
- `Const` makes the reference value read-only.

**If uncommented, this line would cause an error. We cannot modify x.**

```
void increaseValue(const int &x) {
    // x++;
    int y = x + 1;
    cout << "Value after increase: " << y << endl;
}

int main() {
    int x = 10;
    increaseValue(x);
    cout << "Value after increase method: " << x << endl;
}
```

**Console Output:**

```
Value after increase: 11
Value after increase method: 10
```

# Default Arguments

- These are values that are passed *automatically* to a function.
- Might have seen this in Python before.
- Simply, parameters are assigned default values
- If a user does not provide that parameter, *then* the default value is used.

```
bool isEven(int x = 0);
void sayHello(string name, string lastName = "Doe");
```

# Default Arguments

**Example**:

```
bool isEven(int x = 0){
    return x % 2 == 0;
}
```

**Both these calls are valid:**

```
isEven(10);
isEven();
```



Will check if 10 is even



Will check if 0 is even.

# Default Arguments

**Example**:

```
void sayHello(string name, string lastName = "Doe") {
    cout << "Hello " << name << " " << lastName;
}
```

**In this case:**

`name` doesn't have a default value, so it must be given.

`lastName` has a default value so it's optional.

```
sayHello("Ben", "Thomas");
```
← Will print `Hello Ben Thomas`

```
sayHello("John");
```
← Will print `Hello John Doe`

# Default Arguments

**Non-default values must be given first**

```
void sayHello(string name, string lastName = "Doe", int age);
```

✖

**Not valid after previous parameter has a default value.**

```
void sayHello(string name, string lastName = "Doe", int age = 15);
```

✔

```
void sayHello(string name, int age, string lastName = "Doe");
```

✔

**Goes before the default arguments**

# Default Arguments

When we call a function, once we omit a parameter *all* following parameters must be omitted.

```
void sayHello(string name, string lastName = "Doe", int age = 15);
```

```
sayHello("Ben", , 29);
```
✖ **Not valid, after previous parameter was omitted, we must omit all others.**

```
sayHello("Ben");
```
✓

```
sayHello("Ben", "Thomas");
```
✓

# Overloading Functions

- Function Overload: Same function, but different parameter list. name
- Useful when the same task can be done with different inputs.
- Compiler knows which to used based on the parameters given.

```
int add(int, int);
double add(double, double);
int add(int a, int b, int c);
```

# Overloading vs Default Arguments

Instead of writing multiple overloads, we can use **default arguments**.

```
int add(int, int);
int add(int a, int b, int c);
```



```
int add(int a, int b, int c=0);
```

# Returns

# Return Types

- We can return any data type, be it primitive or user defined.
- The `return` keyword can be followed by literals, variables, and expressions.

```
bool isEven(int x = 0){
    return x % 2 == 0;
}
```

```
string getCode(int x = 0){
    if(x == 0)
       return "200 OK";
    else
        return "400 Bad Request";
}
```

# Scope

Life is short...unless you're in an infinite loop

# Scope

- **Scope** = where a variable exists and can be accessed.
- In C++, scope is defined by { }.

**Types of scope**

- **Global** → accessible anywhere in the file.
- **Local** → exists only inside the block it was defined in.

# Scope: Global

- A variable or function created *outside* of any code block.
- Accessible anywhere in the source file, should be avoided.

```
#include <iostream>
int globalValue = 20;
void increaseValueBy(int localValue) {
    globalValue += localValue;
}
int main() {
    std::cout << "Initial globalValue: " << globalValue << std::endl; // Should print 20
    increaseValueBy(5);
    std::cout << "After increaseValueBy(5): " << globalValue << std::endl; // Should print 25
    increaseValueBy(10);
    std::cout << "After increaseValueBy(10): " << globalValue << std::endl; // Should print 35
    return 0;
}
```

**Initialized at file level (not inside any function or class)**

**Can easily be accessed and modified by both functions**

**Console output:**

```
Initial globalValue: 20
After increaseValueBy(5): 25
After increaseValueBy(10): 35
```

# Scope: Local

- A variable or function creates *inside* a code block.
- Inaccessible outside of the code block it was defined in.

The parameters are local variables of this function.

```
int weirdSum(int a, int b) {
    if(a % 2 == 0) {
        int c = 10;
        return a + b + c;
    }
    return a + b;
}
```

The variable c only exists within the if statement.

# Scope: Local

Calling a variable *outside* of its scope would cause an error.

```
int weirdSum(int a, int b) {
    if(a % 2 == 0) {
        int c = 10;
        return a + b + c;
    }
    return a + b + c; ✖ This would throw an error.
}
a = 25; ✖ This would throw an error.
```

# Scope: Local

## Shadowing

- In different scopes you can have variables with the **same name**.
- A local variable can **hide** a global with the same name. Meaning that the local variable takes precedence.

```
int a = 20;
int main() {
    int a = 10;
    std::cout << "Local a: " << a << std::endl;
    std::cout << "Return from sum(a, 5): " << sum(a, 5) << std::endl;
    return 0;
}
```

**Console output:**

```
Local a: 10
Return from sum(a, 5): 15
```

# Static

Affects the lifetime and sharing of variables.

**In C++, we will focus on these contexts**:

- Inside Function (local variable)
- Class (later)

# Static: In Function

- Initializes only once
- Persists between function calls
- Keeps its value until the program ends

**Example use case**: Keeping track of recursive calls

**Keyword static is placed first**

```
void printCounter() {
    static int counter = 0; // static local variable
    counter++;
    std::cout << "Counter: " << counter << std::endl;
}
```

# Static: In Functions

## Example

```
void printCounter() {
    static int counter = 0; // static local variable
    counter++;
    std::cout << "Counter: " << counter << std::endl;
}

int main() {
    for (int i = 0; i < 5; ++i) {
        printCounter();
    }
    return 0;
}
```

Console Output:
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5

# Static: In Functions

## Example

```
void printCounter() {
    static int counter = 0; // static local variable
    counter++;
    std::cout << "Counter: " << counter << std::endl;
}

int main() {
    for (int i = 0; i < 5; ++i) {
        printCounter();
    }
    return 0;
}
```

**Notice that**

- The counter never goes back to 0 (initializes once)
- Each independent call increments the *same* variable (shared variable).

```
Console Output:
Counter: 1
Counter: 2
Counter: 3
Counter: 4
Counter: 5
```

# Conclusion

Your favorite part of any presentation

# Conclusion

- Functions
- Reusable blocks of code → improve modularity & readability.
- Can have parameters (value, reference, const, defaults).
- Can return values (primitives, objects, or nothing with `void`).